

1. Create a python script file called star_wars_pandas.py (15 points)

For this, we will be using the following api:

- i. <https://swapi.dev/api/planets/>
 - ii. no authentication required
 - iii. full api docs here: <https://swapi.dev/>)
- b. Write a function called api_call that takes in a url and makes a GET HTTP request to that url
- i. Take the return data (which is a json string) and **use the json library** to convert it to a python object and name the variable this object is assigned to "json_response"
 - ii. The return of this function should be the json_response object
- c. After the function, initialize the following variables:
- i. planet_list
 1. an empty list
 - ii. diameter
 1. an empty list
 - iii. population
 1. an empty list
 - iv. surface_water
 1. an empty list
 - v. endpoint_url
 1. The string: <https://swapi.dev/api/planets/>
- d. Write a while loop that does the following:
- i. Calls the api_call function passing in the endpoint_url variable and setting the return object to a variable called "results"
 - ii. The data we care about is held in the "results" key of the returned data from the api call. Set the value of the results key to a variable called "planet_info"
 - iii. The results key is a list of dictionaries where each dictionary is planet information. Cycle thru this list and do the following for each planet
 1. If value in the "population" key is "unknown" or "0", stop processing and move to the next planet
 2. If value in the "diameter" key is "unknown" or "0", stop processing and move to the next planet
 3. If value in the "surface" key is "unknown" or "0", stop processing and move to the next planet
 4. Convert the "population" key to an integer and append the value to the population list
 5. Convert the "diameter" key to an integer and append the value to the diameter list

6. Convert the “surface_water” key to a float and append the value to the surface_water list
7. Append the value of the “name” key to the “planet_list
- iv. Once you process all the items in the list, you need to check if there is more data... This api uses pagination where they only send some data at a time. To see if there is more data, you need to check the value of the “next” key in the response from the api.
 1. If the key is none value, then there is no more data.
 - a. You should stop the while loop
 2. If there is more data, the “Next” key will have the next url you should use. If there is another url, you should set this value to “endpoint_url”
 - a. Because you are using a while loop, and we just updated the value of “endpoint_url”, the while loop will start from the top and the api will be called with the new url and you will get the next set of data
- v. Create a panda data from using the following code:

```
data = pd.DataFrame(
    {'population': population,
     'planet_list': planet_list,
     'diameter': diameter,
     'surface_water': surface_water
    })
```

- vi. Create a bubble chart using plotly where x is surface water, y is diameter, size is population, and hover name is planet name
 1. Hint: See “Bubble chart with plotly.express” section here -> <https://plotly.com/python/bubble-charts/>
 2. You should name this figure “fig1”
 3. Run your script and upload a screenshot to this word document of your graph
- vii. BONUS (10 points)
 1. Create a new figure called “fig2” where the data frame is filtered to planets with population less than 27000000000
 2. Run your script and upload a screenshot to this word document of your graph
2. Create a python script file called star_wars_matplotlib.py (15 points)
 - a. Copy all the code from star_wars_pandas.py but delete the import for plotly and the code creating the plotly graphs.
 - b. Create a figure and 3 subplots names ax0, ax1, and ax2 (see matplotlib_5.py for an example)
 - c. Set the following labels:

- i. ax0 x label: Planet
 - ii. ax0 y label: Diameter
 - iii. ax1 x label: Planet
 - iv. ax1 y label: Population
 - v. ax2 x label: Planet
 - vi. ax2 y label: Water Surface
- d. For ax0, plot planet list on x axis, diameter on y axis, and a label of "Diameter"
- e. For ax1, plot planet list on x axis, population on y axis, and a label of "Population"
- f. For ax2, plot planet list on x axis, water surface on y axis, and a label of "Water Surface"
- g. Display the legends for each subplot
- h. Run your script and upload a screenshot to this word document of your graph